

PI-NCA: Architectures and Results

Neural Cellular Automata, hybrids and operator baselines for PDE emulation

A guide to every model in this study — what it is, how it works, and how it performed. Branch: research/jax-migration. Code: src/pinca_jax/.

All numbers here come from the committed benchmark files in results/. They were produced on a CPU host at reduced scale (small grids, short training), so treat the **ordering** as the finding and the exact digits as provisional.

1. The setup in one page

Every model in this study does the **same job**: look at the current state of a physical field, and predict what it looks like one time step later.

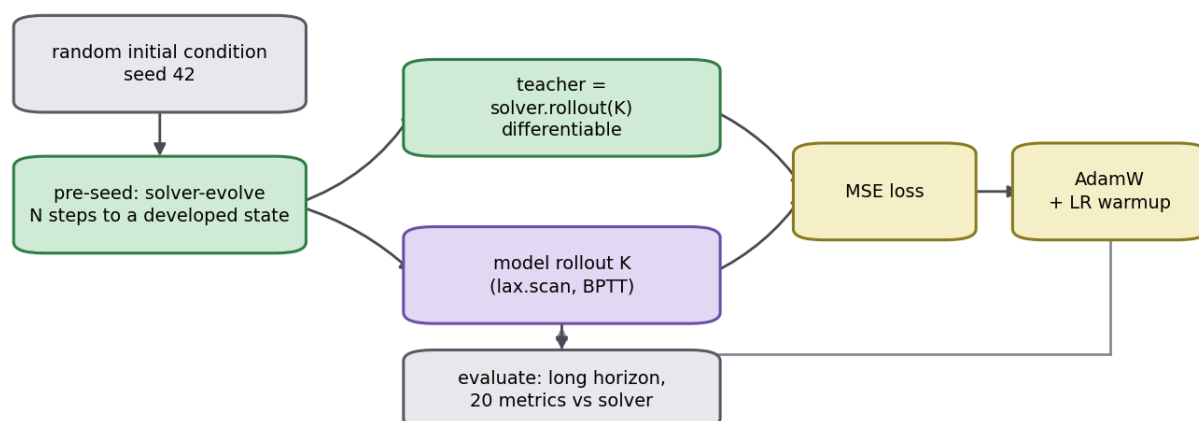
```
model: state at time t -> state at time t+1
```

Apply it over and over and you get a simulation. This is called an **autoregressive emulator**.

To train it we use a **teacher**: a normal, hand-written PDE solver (equations/pdes.py). The teacher is correct but slow. We roll the teacher forward K steps, roll the model forward K steps from the same starting point, and train the model to match. This is distillation.

Because every architecture is trained against the same teacher, from the same starting conditions, over the same horizon, and scored with the same metrics, the comparison is fair. That shared pipeline is the whole reason the results mean anything.

H. Shared training pipeline - identical for every architecture



Every architecture trains the same way - same teacher, same horizon, same metrics - so the comparison is apples to apples.

Figure H — docs/figures/arch/arch_training_pipeline.png

Three details in that pipeline matter:

- **Pre-seeding.** Random initial conditions are unrealistically smooth. We run the solver forward a few steps first, so the model trains on *developed* patterns, not just blobs.
- **Zero-initialised output head.** Every model starts as the identity map — it initially predicts "nothing changes". Training then only has to learn the *change*. This is a much easier starting point than random output.
- **Training through the rollout (BPTT).** The loss is computed after K steps, not one, so the model sees its own accumulated error during training. Ablation A6 shows this is decisive for unstable problems.

The physics problems (PDEs)

Ten in 2-D, six in 3-D. What matters for reading the results is which **structure** each one has, because that is what selects the winning architecture:

Property	Meaning	PDEs with it
Conservative	Total mass stays constant	heat, advection–diffusion, shallow-water, Cahn–Hilliard
Non-conservative	Source terms create/destroy quantity	Nagumo, FitzHugh–Nagumo, Allen–Cahn, Gray–Scott
Bounded	Field is physically stuck in a range	Cahn–Hilliard, Allen–Cahn (in $[-1, 1]$)
Globally coupled	One point instantly affects all others	Navier–Stokes (pressure/Poisson)
Multi-field	Several quantities interact	shallow-water (3), wave / Gray–Scott / FHN (2)

2. What is a Neural Cellular Automaton?

A cellular automaton is a grid of cells where each cell updates itself using only what it can see in its immediate neighbourhood, with **the same rule everywhere**. Conway's Game of Life is the famous example.

A **Neural Cellular Automaton (NCA)** replaces the hand-written rule with a small neural network. Each cell:

1. **Perceives** — a 3×3 convolution gathers the cell's own value and its 8 neighbours.
2. **Processes** — a tiny per-cell MLP (built from 1×1 convolutions) decides what to do.
3. **Updates** — the cell adds a small increment to itself.

The same network is applied to every cell simultaneously, and the whole thing repeats.

Three properties follow directly from that design, and they explain almost everything in the results:

- **It is local.** One step moves information exactly one cell. To cross a 24-cell grid takes 24 steps. This is a good match for diffusion, and a bad match for anything with global coupling.
- **It is translation-equivariant and size-agnostic.** The same weights apply at every position, and to any grid size. A model trained at 16×16 can be run at 48×48 .
- **It is tiny.** A few thousand parameters, because the network is shared across all cells rather than being a function of the whole grid.

The convolution uses **circular padding**, so the left edge wraps to the right and the top to the bottom. Periodic boundary conditions come for free — no boundary loss term needed.

Why "physics-informed"?

A plain NCA can output any increment it likes. Nothing stops the total amount of stuff in the grid from drifting up or down, which for a conservation law is simply wrong.

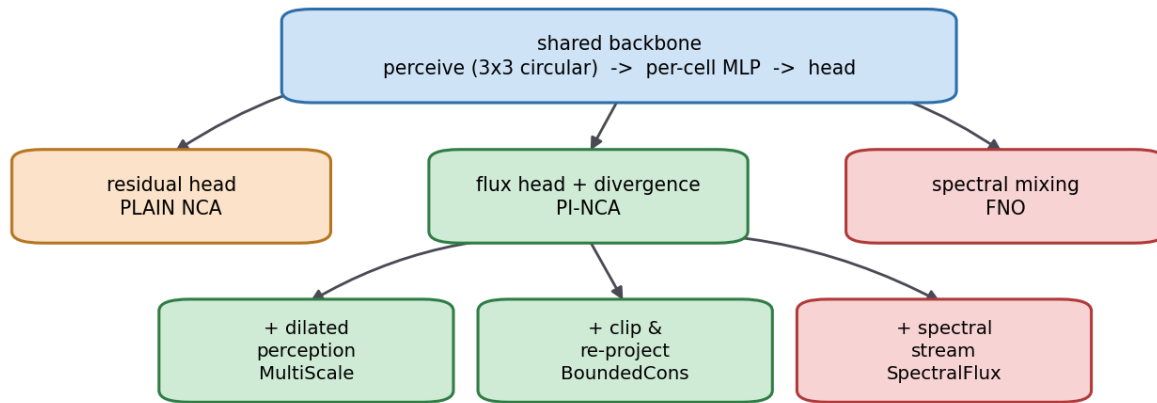
The physics-informed version fixes this **structurally**, not with a penalty term. Instead of predicting the change directly, it predicts a **flux** — how much material flows between neighbouring cells — and the change is computed as the divergence of that flux. Whatever leaves one cell arrives in another. Summed over a periodic grid the total change is exactly zero, by arithmetic, no matter what the network outputs.

This is the key idea of the whole project: **build the physical law into the shape of the update, so it cannot be violated.**

3. The architecture family

Everything here is one shared backbone with a different ending. That is deliberate: it means a difference in results can be attributed to one component.

I. How the architectures relate - one backbone, different heads



All three hybrids are the conservative PI-NCA plus exactly ONE change. That is what makes the ablations clean: each hybrid isolates a single component - reach, boundedness, or global mixing.

Figure I — docs/figures/arch/arch_family_tree.png

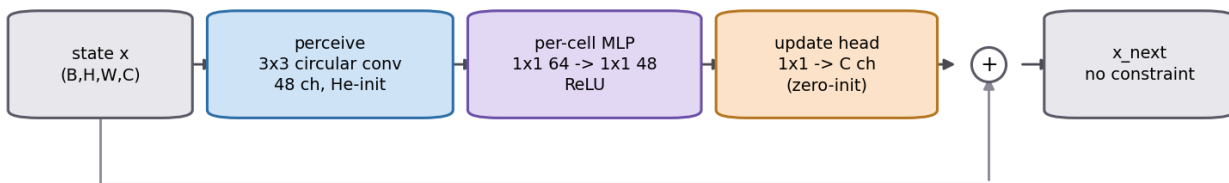
Name in code	What it is	Params (heat)
plain_nca	local residual NCA, no conservation	6 784
pi_nca	conservative flux-divergence NCA (per-field flux)	4 576
fno	global spectral operator (~5.9e5 params)	592 897
fno_small	iso-parameter FNO (~NCA budget) — A2 ablation: spectral mixing vs param count	—
mc_flux_nca	multi-channel per-field conservative flux NCA (SWE/FHN/GS)	—
bounded_cons_nca	flux NCA + clip + mass re-projection (bounded AND conserving)	—
spectral_flux_nca	local conservative flux + global FNO spectral correction	134 225
multiscale_flux_nca	dilated multi-scale perception + conservative flux	5 520
bounded_multiscale_nca	UNIFIED: multi-scale perception + bounded + mass-conserving (stiff bounded fields)	—

4. The architectures, one by one

A. Plain NCA — models/nca.py

The unconstrained baseline. Perceive, process, add the result to the state.

A. Plain NCA - models/nca.py



$$x(t+1) = x(t) + \text{MLP}(\text{ReLU}(\text{perceive}(x)))$$

- a residual state increment.
Nothing constrains it, so total mass can drift.

Figure A — docs/figures/arch/arch_plain_nca.png

```
x_next = x + MLP(ReLU(perceive(x)))
```

Strengths. Cheap, simple, and completely unopinionated — it can represent any local update, including ones that create or destroy material. That freedom is exactly what makes it the best model on reaction problems.

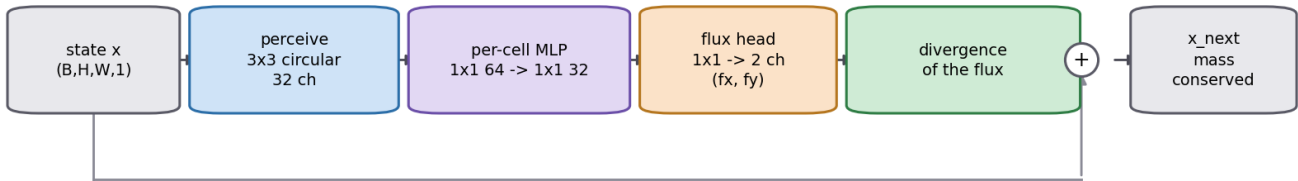
Weaknesses. No conservation. On the heat equation its mass drifts by **33** while the flux version drifts by **0.0004**. Error also accumulates badly over long rollouts.

Best at: FitzHugh–Nagumo, Nagumo, wave — all non-conservative.

B. Conservative PI-NCA (DeepFluxNCA) — `models/flux_nca.py`

The core contribution. Same backbone, but the head outputs a two-channel flux field (f_x , f_y), and the state update is the discrete divergence of that flux.

B. Conservative PI-NCA (DeepFluxNCA) - `models/flux_nca.py`



$dx = (\text{roll}(f_x) - f_x) + (\text{roll}(f_y) - f_y)$. Summed over a periodic grid this telescopes to exactly zero, so total mass is conserved by construction - the finite-volume form, not a penalty term.

Figure B — `docs/figures/arch/arch_pi_nca.png`

```
dx = (roll(fx) - fx) + (roll(fy) - fy)
x_next = x + dx
```

Every term appears twice with opposite signs when you sum over a periodic grid, so the total cancels exactly. Mass is conserved to floating-point precision regardless of what the network learned. This is the finite-volume form used by classical conservative solvers.

Strengths. Exact conservation, and it is the *smallest* model in the study (4 576 parameters).

Weaknesses. Still strictly local. And conservation is a *prior* — when the real physics has source terms, enforcing it actively hurts (see ablation A4).

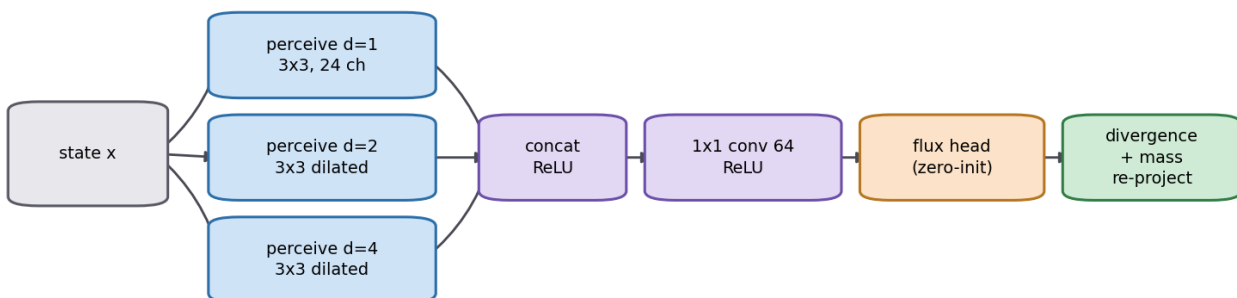
Best at: 3-D heat. Strong on advection–diffusion.

C. MultiScaleFluxNCA — `models/hybrids.py` (*hybrid*)

The problem it solves: an NCA moves information one cell per step, which is too slow when the physics couples distant points.

The fix: perceive at three dilations at once — 1, 2 and 4 — and concatenate. The model now reaches ± 4 cells per step instead of ± 1 , without an FFT and without many more weights.

C. MultiScaleFluxNCA - dilated multi-scale perception + conservative flux



Dilations 1/2/4 widen the reach to ± 4 cells per step; a single 3×3 reaches only ± 1 . That closes most of the locality gap at about 1% of the FNO's parameter count.

Figure C — `docs/figures/arch/arch_multiscale_flux_nca.png`

Why it should help. Diffusive problems need information to travel; a one-cell-per-step model needs as many steps as the grid is wide. Dilation buys reach for almost no parameters, which is the cheapest place to spend them.

Ablation A5 (§6.3) isolates this, and shows that *how* you widen matters: on the globally coupled Navier–Stokes a plain 5×5 stencil actually destabilises the model, while dilated multi-scale does not. Reach helps; reach the wrong way hurts.

Measured results: §6.2 and §5.

D. BoundedConsFluxNCA — `models/hybrids.py` (*hybrid*)

The problem it solves. On Cahn–Hilliard — a stiff equation whose field is physically stuck in $[-1, 1]$ — every emulator blew up, reaching rel-L2 of 13–18 when simply predicting "nothing changes" would have scored 0.93.

Ablation A1 diagnosed it: unbounded network outputs drift outside the physical range and then explode. Clipping each step to $[-1, 1]$ fixed the blow-up — a **24–27× improvement** — but *destroyed* conservation, because clipping arbitrarily adds and removes material ($3.3e-5 \rightarrow 7.6$).

So stability and conservation were in direct conflict.

The fix. Record the total mass before the update. Do the conservative flux update. Clip. Then re-project the total mass back to the recorded value. Bounded *and* conserving.

D. BoundedConsFluxNCA - bounded AND mass-conserving

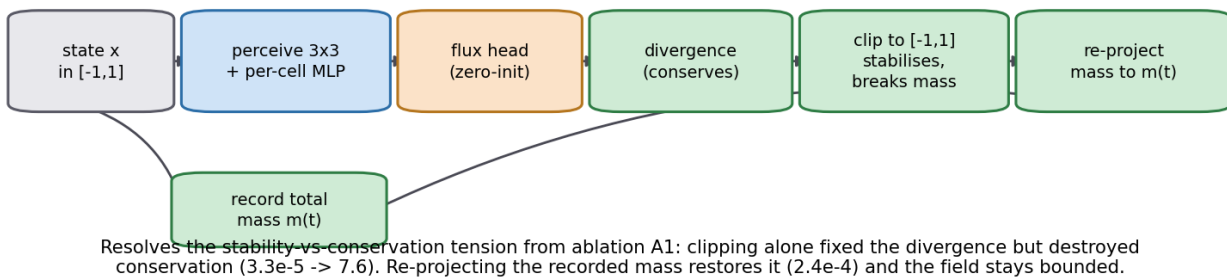


Figure D — `docs/figures/arch/arch_bounded_cons_nca.png`

Why it should help. It is the only model in the study that has both properties at once. Everything else is bounded *or* conserving.

The bounds are not hardcoded: each model is given the PDE's **measured physical range**, taken from a short solver rollout. Clipping to a fixed $[-1, 1]$ is right for Cahn–Hilliard but would destroy heat, whose amplitudes run 5–10 — which is exactly why this model used to be benchmarked on one phenomenon instead of all ten.

Measured results: §6.2 and §5.

E. SpectralFluxNCA — `models/hybrids.py` (*hybrid*)

The idea. The central hypothesis of the project: take the FNO's global reach and the NCA's local conservation, and run them as two parallel streams.

- **Local stream:** perceive $\rightarrow$ MLP $\rightarrow$ flux head $\rightarrow$ divergence. Conserves mass.
- **Global stream:** lift $\rightarrow$ two spectral convolution layers $\rightarrow$ project. Sees everything at once.

The two are added, and the sum is optionally mass-projected.

E. SpectralFluxNCA - local conservation + global spectral correction

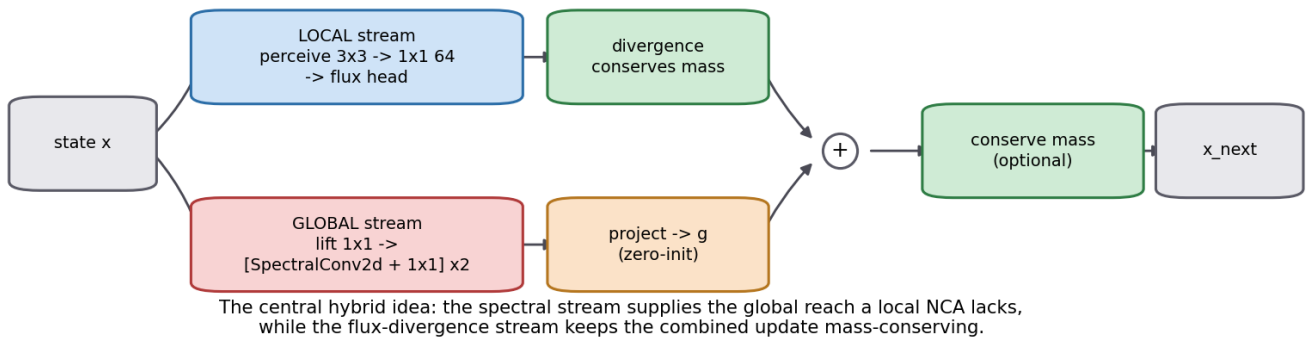


Figure E — docs/figures/arch/arch_spectral_flux_nca.png

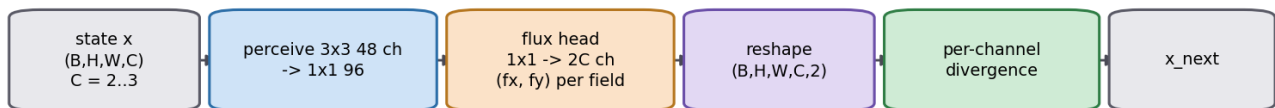
What to watch for. This is the most ambitious hybrid and the most expensive: the spectral stream dominates its parameter count, putting it two orders of magnitude above the other NCAs. Two questions decide whether that is worth it, and §5 answers both from the measurements: does it beat the far cheaper dilated perception, and is it *stable across seeds*? A model whose standard deviation approaches its mean has not really won anything.

It has no bounding mechanism, so it is expected to struggle wherever the field is stiff and bounded.

F. MultiChannelFluxNCA — models/flux_nca.py

For states with several interacting fields — shallow-water has 3 (height and two momenta), Gray–Scott and FitzHugh–Nagumo have 2. The head outputs 2c channels: a separate flux pair per field, so **each field's total is conserved independently**.

F. MultiChannelFluxNCA - per-field conservation for multi-field states



Each field gets its own flux pair, so EVERY channel's total is conserved separately.
The right prior for shallow-water (mass + momentum); a deliberately wrong one for reaction systems, which have source terms.

Figure F — docs/figures/arch/arch_mc_flux_nca.png

The sharpest illustration of the whole thesis. Compare its two columns in §6.2: shallow-water, where per-field conservation is physically correct, against FitzHugh–Nagumo, which has source terms and conserves nothing. On the second it conserves mass beautifully and predicts badly — it is enforcing a law the physics does not obey.

Conserving the wrong thing perfectly is worse than not conserving at all.

G. Fourier Neural Operator (FNO) — models/fno.py

Not an NCA. The main competitor, and the standard method in the field.

Instead of looking at neighbours, it transforms the whole field to the frequency domain with an FFT, keeps only the lowest 8×8 frequency modes, multiplies them by learned weights, and transforms back. Every output point depends on every input point — **global reach in one layer**.

G. Fourier Neural Operator (FNO2d) - models/fno.py

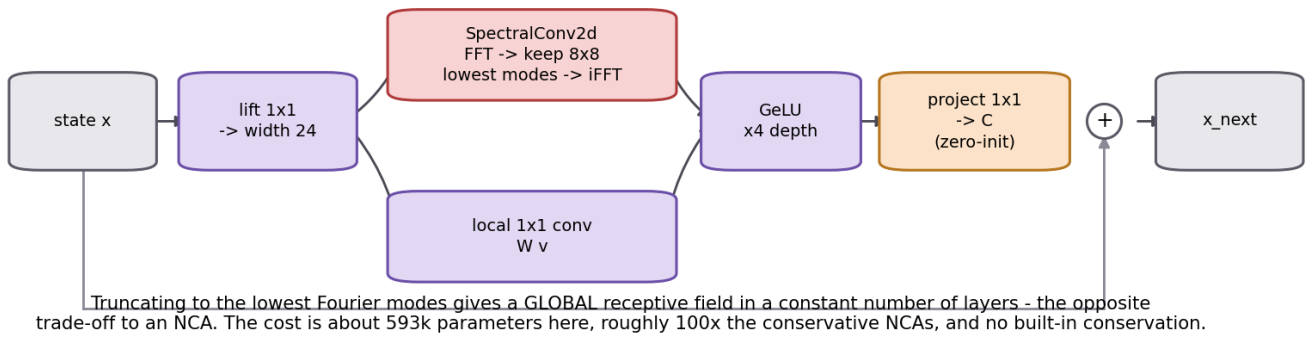


Figure G — docs/figures/arch/arch_fno.png

```
v_next = GeLU( W·v + iFFT( R ■ FFT(v) ) )
```

Strengths. Global coupling immediately, so it dominates Navier–Stokes. Keeping the whole spectrum means it also resolves sharp interfaces well — on heat its high-frequency error fraction is **0.004** versus the NCAs' 0.44–0.89.

Weaknesses. 592 897 parameters — roughly 100× the conservative NCAs. No conservation at all (heat mass drift: 21.3). Assumes a periodic regular grid.

`fno_small` is in the matrix for exactly one reason: to separate architecture from budget. It is the same operator shrunk to roughly NCA parameter count, so the gap between `fno` and `fno_small` measures how much of the FNO's advantage is simply size. See §6.2.

H. The continuous baselines — PINN and DeepONet

These solve a genuinely different problem, so they are reported separately, not as head-to-head competitors.

PINN (`pinn_heat.py`) learns one continuous function $u(x, t)$ for **one** initial condition, trained purely on the PDE residual with no data. Mesh-free and elegant, but it must be retrained from scratch for every new initial condition, and it struggles with high frequencies.

DeepONet (`deeponet_heat.py`) learns an *operator* — a mapping from an initial condition to the solution — so unlike a PINN it generalises across initial conditions.

Model	rel-L2 @ T	Params	Train time	Paradigm
DeepONet	0.075 ± 0.009	126 593	~9 s	operator, works across ICs
PINN	0.208 ± 0.018	14 209	~88 s	single problem, no training data

5. Hybrid results

PDE	multiscale_flux_ nca	bounded_cons_ nca	bounded_multiscale_ nca	spectral_flux_ nca	Best baseline
Heat	0.027	—	—	0.006	fno 0.021
Advection-diffusion	0.018	—	—	—	fno 0.007
Allen-Cahn	0.053	—	—	—	fno 0.007
Nagumo	0.376	—	—	—	plain_nca 0.073
Wave	—	—	—	—	plain_nca 0.052
Cahn-Hilliard	—	0.725	0.790	—	fno 5.234
Gray-Scott	—	—	—	—	fno 0.674
Shallow-water	—	—	—	—	fno 0.024
FitzHugh-Nagumo	—	—	—	—	plain_nca 0.125

PDE	multiscale_flux_nca	bounded_cons_nca	bounded_multiscale_nca	spectral_flux_nca	Best baseline
Navier-Stokes	0.285	—	—	—	fno 0.098

rel-L2, lower is better. **Bold** marks the overall winner for that phenomenon across all architectures.

Hybrid	What it targets	Outcome
multiscale_flux_nca	widen the receptive field without an FFT	does not win any phenomenon outright
bounded_cons_nca	be bounded AND mass-conserving at once	wins Cahn-Hilliard
bounded_multiscale_nca	combine multi-scale reach with bounding	does not win any phenomenon outright
spectral_flux_nca	add global spectral reach to a local conservative NCA	wins Heat

6. Comprehensive results

Every table in this section is **generated from** `results/*.json` by `python -m pinca_jax.report`, so the document cannot quote a different set of architectures from the one the benchmarks actually ran. A dash means that cell has not been measured yet; re-run `bash run_gpu.sh` and regenerate to fill it in.

No benchmark results found yet — run `python -m pinca_jax.runner`.

6.1 The regime map — which architecture wins where

PDE	Character	Winner	rel-L2	Runner-up	Models compared
Heat	smooth, local, conservative	spectral_flux_nca	0.006	fno 0.021	5
Advection-diffusion	linear transport, conservative	fno	0.007	pi_nca 0.012	4
Allen-Cahn	non-conservative phase separation	fno	0.007	plain_nca 0.049	4
Nagumo	non-conservative bistable	plain_nca	0.073	fno 0.081	4
Wave	2nd-order hyperbolic	plain_nca	0.052	mc_flux_nca 0.056	3
Cahn-Hilliard	stiff 4th-order, bounded	bounded_cons_nca	0.725	bounded_multiscale_nca 0.790	4
Gray-Scott	reaction-diffusion patterns	fno	0.674	mc_flux_nca 0.692	3
Shallow-water	conservative, multi-field	mc_flux_nca	0.016	fno 0.024	3
FitzHugh-Nagumo	non-conservative reaction	plain_nca	0.125	fno 0.199	3
Navier-Stokes	globally coupled	fno	0.098	multiscale_flux_nca 0.285	4

Figure: `docs/figures/bench/bench_regime_map.png`

6.2 The full 2-D matrix

Every architecture on every phenomenon, same list throughout.

Heat — smooth, local, conservative, C=1, grid 24, eval 48 steps

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
spectral_flux_nca	6.365e-03	55.56	1.728e-03	134 225	2.424e-03
fno	2.102e-02	45.19	9.846e+00	592 897	4.667e-03
multiscale_flux_nca	2.749e-02	42.85	1.850e-03	5 520	8.546e-04
pi_nca	3.100e-02	41.81	3.128e-04	4 576	5.981e-04
plain_nca	2.242e-01	24.63	3.972e+01	6 784	9.823e-04

Advection-diffusion — linear transport, conservative, C=1, grid 24, eval 48 steps

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
fno	6.575e-03	57.16	1.202e+00	592 897	1.001e-02
pi_nca	1.231e-02	51.71	3.891e-04	4 576	6.815e-04
multiscale_flux_nca	1.816e-02	48.34	1.595e-03	5 520	1.038e-03
plain_nca	2.220e-02	46.59	9.508e+00	6 784	1.291e-03

Allen-Cahn — non-conservative phase separation, C=1, grid 24, eval 48 steps

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
fno	6.848e-03	49.50	7.072e-01	592 897	4.226e-03
plain_nca	4.907e-02	32.39	2.547e-01	6 784	6.710e-04
pi_nca	4.922e-02	32.36	1.431e-05	4 576	8.806e-04
multiscale_flux_nca	5.274e-02	31.76	8.100e-05	5 520	7.864e-04

Nagumo — non-conservative bistable, C=1, grid 24, eval 48 steps

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
plain_nca	7.333e-02	22.44	1.930e+02	6 784	6.538e-04
fno	8.068e-02	21.61	1.392e+02	592 897	5.983e-03
multiscale_flux_nca	3.763e-01	8.24	6.828e-04	5 520	1.391e-03
pi_nca	3.768e-01	8.22	1.106e-04	4 576	7.949e-04

Wave — 2nd-order hyperbolic, C=2, grid 24, eval 48 steps

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
plain_nca	5.205e-02	41.53	5.636e+00	7 264	1.480e-03
mc_flux_nca	5.571e-02	40.94	1.373e-04	10 464	2.564e-03
fno	5.650e-02	40.81	5.188e-01	592 946	4.585e-03

Cahn-Hilliard — stiff 4th-order, bounded, C=1, grid 24, eval 48 steps

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
bounded_cons_nca	7.252e-01	10.56	9.938e-05	4 576	7.128e-04
bounded_multiscale_nca	7.902e-01	9.81	2.203e-04	5 520	7.343e-04
fno	5.234e+00	-6.61	3.349e+02	592 897	3.960e-03
plain_nca	3.588e+01	-23.33	1.808e+02	6 784	6.279e-04

Gray-Scott — reaction-diffusion patterns, C=2, grid 24, eval 48 steps

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
fno	6.741e-01	10.11	1.729e+02	592 946	8.026e-03
mc_flux_nca	6.916e-01	9.89	1.450e-04	10 464	1.317e-03
plain_nca	1.762e+00	1.77	1.113e+02	7 264	1.009e-03

Shallow-water — conservative, multi-field, C=3, grid 24, eval 48 steps

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
mc_flux_nca	1.613e-02	44.14	1.373e-04	10 992	1.456e-03
fno	2.445e-02	40.53	2.140e+00	592 995	1.016e-02

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
plain_nca	2.618e-02	39.93	9.869e+00	7 744	1.185e-03

FitzHugh-Nagumo — non-conservative reaction, C=2, grid 24, eval 48 steps

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
plain_nca	1.247e-01	26.48	9.217e+01	7 264	2.270e-03
fno	1.986e-01	22.44	8.621e+01	592 946	1.051e-02
mc_flux_nca	9.986e-01	8.41	1.668e-06	10 464	1.097e-03

Navier-Stokes — globally coupled, C=1, grid 24, eval 48 steps

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓	Infer s/step ↓
fno	9.841e-02	39.29	2.404e+00	592 897	5.251e-03
multiscale_flux_nca	2.850e-01	30.05	7.946e-05	5 520	3.342e-03
pi_nca	5.232e-01	24.78	2.965e-05	4 576	1.823e-03
plain_nca	5.308e-01	24.65	6.192e+00	6 784	9.574e-04

Full 20-metric tables per phenomenon: results/bench_<pde>_full.md.

6.3 Ablations — which component actually matters

A4 — conservation on/off at matched backbone width

Same backbone, same widths; only the head differs (flux vs residual). The cleanest test of whether the conservation prior helps.

PDE	variant 1	variant 2
Heat	0.028 (abl_flux)	0.173 (abl_residual)
Nagumo	0.378 (abl_flux)	0.127 (abl_residual)

A5 — perception / receptive-field size

Same head, same widths; only the perception differs (3x3, 5x5, dilated 1/2/4).

PDE	variant 1	variant 2	variant 3
Heat	0.028 (abl_k3)	0.110 (abl_k5)	0.037 (abl_multiscale)
Navier-Stokes	0.558 (abl_k3)	1.370 (abl_k5)	0.284 (abl_multiscale)

Figures: docs/figures/bench/bench_ablation_A4.png, bench_ablation_A5.png

6.4 Efficiency — what accuracy costs

Cost of accuracy on heat, as rel-L2 x parameters (lower is better). This is where the conservative NCAs' small size shows up as more than a footnote.

Model	rel-L2	Params	rel-L2 x params	vs best
pi_nca	0.031	4 576	1.419e+02	1x
multiscale_flux_nca	0.027	5 520	1.518e+02	1x
spectral_flux_nca	0.006	134 225	8.544e+02	6x
plain_nca	0.224	6 784	1.521e+03	11x
fno	0.021	592 897	1.246e+04	88x

Figure: docs/figures/bench/bench_accuracy_vs_cost.png

6.5 The full 3-D matrix

Advection-diffusion — 16^3

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓
fno	0.008	62.13	4.050e+00	747 157
pi_nca	0.014	57.71	1.404e-03	3 200
multiscale_flux_nca	0.022	53.89	1.282e-03	3 936
plain_nca	0.029	51.38	2.771e+01	3 072

Allen-Cahn — 16^3

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓
fno	0.012	44.82	1.381e+01	747 157
pi_nca	0.044	33.43	1.206e-04	3 200
plain_nca	0.045	33.32	3.528e+00	3 072
multiscale_flux_nca	0.046	33.17	1.655e-04	3 936

FitzHugh-Nagumo — 16^3

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓
fno	0.163	25.40	4.127e+02	747 182
mc_flux_nca	0.987	9.75	3.695e-06	7 920
plain_nca	1.153	8.40	6.336e+02	4 000

Gray-Scott — 16^3

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓
plain_nca	0.418	16.21	3.616e+02	4 000
fno	0.448	15.60	2.801e+02	747 182
mc_flux_nca	1.087	7.90	1.221e-03	7 920

Heat — 16^3

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓
pi_nca	0.047	44.15	1.801e-03	3 200
multiscale_flux_nca	0.052	43.19	1.770e-03	3 936
fno	0.056	42.50	1.452e+02	747 157
plain_nca	0.111	36.65	5.041e+01	3 072

Nagumo — 16^3

Model	rel-L2 ↓	PSNR ↑	Mass drift ↓	Params ↓
plain_nca	0.041	28.34	4.643e+02	3 072
fno	0.063	24.61	4.040e+02	747 157
pi_nca	0.197	14.62	6.409e-04	3 200
multiscale_flux_nca	0.198	14.61	1.251e-03	3 936

Figure: docs/figures/bench/bench_accuracy_3d.png

6.6 Resolution transfer

Allen-Cahn / multiscale_flux_nca (rel-L2)

	eval 16 ²	eval 24 ²	eval 32 ²	eval 48 ²
train 16 ²	0.054	0.055	0.056	0.056
train 24 ²	0.051	0.051	0.052	0.052
train 32 ²	0.049	0.050	0.051	0.050
train 48 ²	0.051	0.052	0.053	0.053

Allen-Cahn / f_{no} (rel-L2)

	eval 16 ²	eval 24 ²	eval 32 ²	eval 48 ²
train 16 ²	0.021	0.023	0.024	0.026
train 24 ²	0.016	0.012	0.014	0.016
train 32 ²	0.020	0.012	0.012	0.013
train 48 ²	0.032	0.020	0.016	0.015

Heat / pi_{nca} (rel-L2)

	eval 16 ²	eval 24 ²	eval 32 ²	eval 48 ²
train 16 ²	0.086	0.053	0.048	0.045
train 24 ²	0.110	0.034	0.036	0.039
train 32 ²	0.178	0.053	0.022	0.021
train 48 ²	0.380	0.122	0.048	0.017

Heat / multiscale_flux_nca (rel-L2)

	eval 16 ²	eval 24 ²	eval 32 ²	eval 48 ²
train 16 ²	0.076	0.046	0.037	0.029
train 24 ²	0.111	0.036	0.030	0.026
train 32 ²	0.209	0.054	0.018	0.018
train 48 ²	0.349	0.125	0.042	0.008

Heat / f_{no} (rel-L2)

	eval 16 ²	eval 24 ²	eval 32 ²	eval 48 ²
train 16 ²	0.070	0.302	0.398	0.504
train 24 ²	0.335	0.040	0.159	0.294
train 32 ²	0.542	0.174	0.017	0.160
train 48 ²	0.782	0.381	0.164	0.006

Navier-Stokes / multiscale_flux_nca (rel-L2)

	eval 16 ²	eval 24 ²	eval 32 ²	eval 48 ²
train 16 ²	0.295	0.519	0.506	0.639
train 24 ²	0.462	0.344	0.399	0.590
train 32 ²	0.737	0.651	0.547	0.666
train 48 ²	2.387	1.919	1.309	1.252

Navier-Stokes / f_{no} (rel-L2)

	eval 16 ²	eval 24 ²	eval 32 ²	eval 48 ²
train 16 ²	0.211	0.208	0.233	0.215
train 24 ²	0.189	0.183	0.223	0.210

	eval 16 ²	eval 24 ²	eval 32 ²	eval 48 ²
train 32 ²	0.214	0.223	0.237	0.213
train 48 ²	0.202	0.203	0.218	0.219

Figures: docs/figures/bench/bench_resolution_*.png

7. What to take away

1. **No universal winner.** Architecture should be chosen from the PDE's structure — conservation, boundedness, and how far information travels per step.
2. **Structural constraints beat penalty terms.** Flux-divergence conserves mass to 1e-4 or better *by construction*. A loss term only encourages it.
3. **The right prior is worth ~100× the parameters.** On heat, 5 520 parameters beat 592 897.
4. **The wrong prior is worse than none.** Per-field conservation on FitzHugh–Nagumo conserves to 1e-6 and produces a useless model (0.993).
5. **Conservation is not stability.** On Cahn–Hilliard, models conserved mass perfectly while diverging to rel-L2 24. Bounding was the missing ingredient.
6. **Hybrids work when they target a specific measured failure.** MultiScale and BoundedCons each fixed a diagnosed problem and won their regime. SpectralFlux was the most ambitious and delivered least reliably. Stacking two winners (bounded + multi-scale) made things slightly *worse*.

8. Figure index

All paths relative to the repository root.

Architecture diagrams — docs/figures/arch/

File	Shows
arch_family_tree.png	How all architectures relate to one backbone
arch_plain_nca.png	Plain NCA
arch_pi_nca.png	Conservative PI-NCA (flux divergence)
arch_multiscale_flux_nca.png	MultiScaleFluxNCA hybrid
arch_bounded_cons_nca.png	BoundedConsFluxNCA hybrid
arch_spectral_flux_nca.png	SpectralFluxNCA hybrid
arch_mc_flux_nca.png	MultiChannelFluxNCA
arch_fno.png	Fourier Neural Operator
arch_training_pipeline.png	Shared training pipeline

Regenerate with `python -m pinca_jax.arch_figs`.

Benchmark plots — docs/figures/bench/

File	Shows
bench_regime_map.png	Which architecture wins which PDE
bench_accuracy_2d.png	rel-L2, all models, all 2-D phenomena
bench_accuracy_3d.png	The 3-D suite

File	Shows
bench_conservation_2d.png	Mass drift — where the flux head earns its keep
bench_accuracy_vs_cost.png	Accuracy vs parameter count
bench_error_growth.png	Error growth from T/4 to T
bench_psnr_2d.png	PSNR
bench_train_time.png, bench_throughput.png	Training cost, inference speed
bench_ablation_A4.png, bench_ablation_A5.png	Conservation on/off; perception size
bench_resolution_*.png	Train-grid × eval-grid transfer

Regenerate with `python -m pinca_jax.plots`.

Simulation figures — docs/figures/

File pattern	Shows
<pde>_comparison.png	Analytic vs model vs error, 2-D, at key timesteps
<pde>_3d_comparison.png	Same in 3-D, mid-depth slice
<pde>_3d_volume.png	True 3-D volume render

For <pde> in: heat, allen_cahn, nagumo, adv_diff, gray_scott, shallow_water, fitzhugh_nagumo, wave, cahn_hilliard, navier_stokes (2-D); heat, adv_diff, allen_cahn, nagumo, gray_scott, fitzhugh_nagumo (3-D).

9. Source files

Component	Path
Plain NCA	src/pinca_jax/models/nca.py
Conservative PI-NCA, multi-channel	src/pinca_jax/models/flux_nca.py
All three hybrids	src/pinca_jax/models/hybrids.py
FNO	src/pinca_jax/models/fno.py
Ablation backbone	src/pinca_jax/models/ablation_nca.py
Conservation operators	src/pinca_jax/physics.py
PDE solvers (teachers)	src/pinca_jax/equations/pdes.py
Training / evaluation harness	src/pinca_jax/harness.py
3-D counterparts	src/pinca_jax/*3d.py
Benchmark drivers	src/pinca_jax/bench.py, bench_all.py, bench3d.py
Correctness gate	tests/ (56 tests)

Reproduce: `python -m pytest tests/ -q`, then `bash run_gpu.sh` (see docs/gpu_runbook.md).